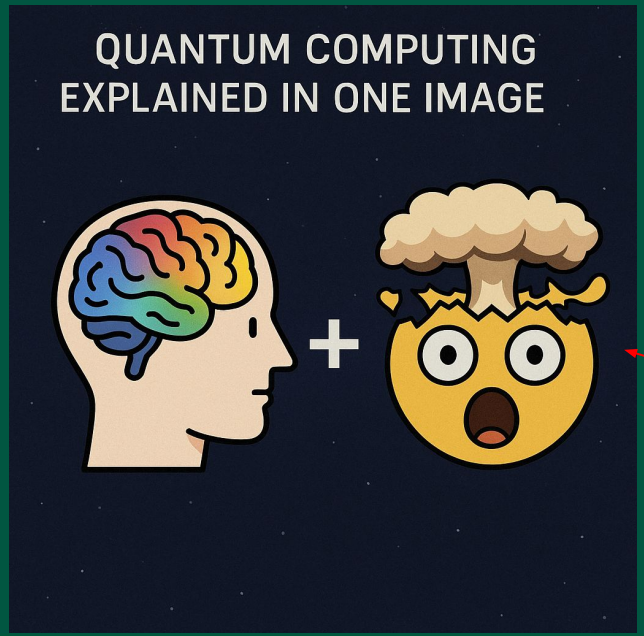
# QML TORCH




## Charlotte AI Research

Summer Malik

you after you hear about qml torch

THE WILLIAM STATES LEE
COLLEGE OF ENGINEERING

# What is QML Torch?

**QML Torch is an open-source framework that integrates quantum computing into machine learning workflows using PyTorch.**

When we train models today, they're strictly classical each layer either activates or it doesn't, each parameter either shifts or it doesn't. Quantum machine learning challenges this binary view by introducing qubits that can exist in overlapping states, creating fundamentally new ways to represent and process information.

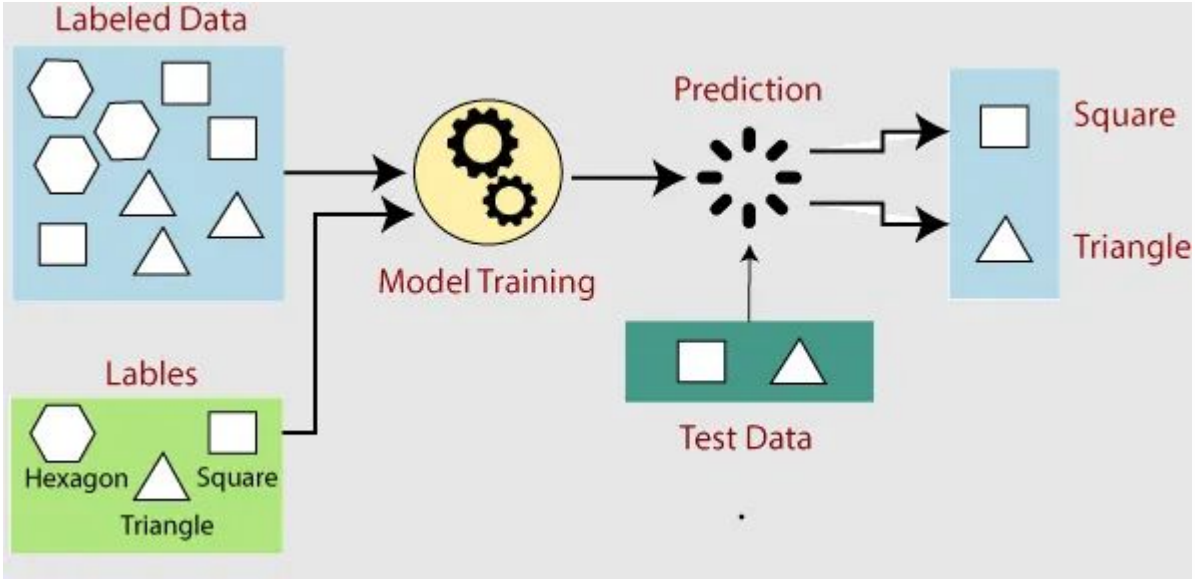
Existing tools like PennyLane are powerful but complex to set up and integrate with modern machine learning pipelines. QML Torch aims to change this by providing a simple `import qmltorch` interface inside PyTorch, making it possible to insert quantum layers directly into neural networks without heavy boilerplate.

## QML Torch = making quantum machine learning as easy as adding a new layer in PyTorch.

# SO… what even is machine learning?

**When we talk about machine learning today, everything runs on classical computers**

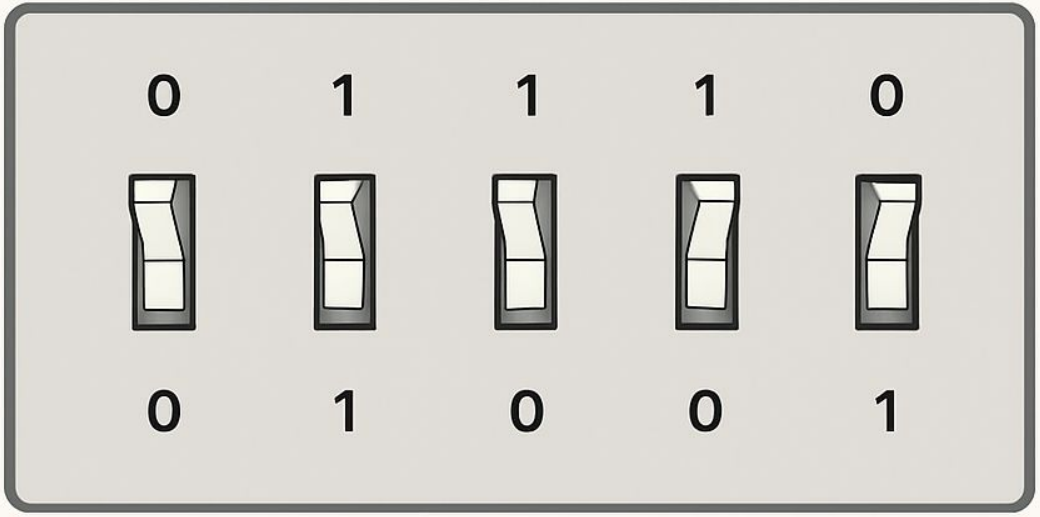
**The data goes in the model→ the model makes a prediction —-> we adjust the parameters until it gets better**




**It's powerful, but everything is built from the same building blocks: classical bits (0 or 1).**

This poses issues because a bit can only be in 1 of 2 states at the same time.

# Classical Bits (0 or 1)




This is how classical computers store and process information



Quantum bits (qubits) aren't just on or off... they can be somewhere in between

# What does Quantum add?

## Classic Machine Learning

- Tries one guess (weight setting) at a time → like opening one door at a time
- Features are compared separately → like two friends taking independent notes.
- Combines results step by step → like adding votes in a normal tally.
- Sampling complex probabilities is slow → like rolling lots of dice until you see a rare outcome.

## Quantum Machine Learning

- Tries many guesses at once in superposition → like peeking behind many doors at once.
- Features can be entangled and linked → like friends who always finish each other's sentences, no matter the distance.
- Cancels bad paths and strengthens good ones → like a chorus canceling wrong notes and amplifying harmony.
- Naturally samples from complex distributions → like having dice weighted perfectly for the distribution you need.

# QML without QML torch

## PyTorch Model Layer Today (typical PennyLane + PyTorch integration) — verbose, lots of boilerplate

```python
import torch
import pennylane as qml

# Device + circuit
dev = qml.device("default.qubit", wires=4)

def encode_angle(x):
    for i in range(4):
        qml.RY(x[i], wires=i)

def ansatz(params):
    # e.g., RY layer + entanglement
    for i in range(4):
        qml.RY(params[i], wires=i)
    for i in range(3):
        qml.CNOT(wires=[i, i+1])

@qml.qnode(dev, interface="torch", diff_method="best")
def quantum_block(x, params):
    encode_angle(x)
    ansatz(params)
    return qml.expval(qml.PauliZ(0))

class HybridModel(torch.nn.Module):
    def __init__(self):
        super().__init__()
        self.pre = torch.nn.Linear(4, 4)
        self.theta = torch.nn.Parameter(torch.zeros(4))
    def forward(self, x):
        x = self.pre(x)
        # Per-sample call; batching needs custom handling
        outs = [quantum_block(xi, self.theta) for xi in x]
        return torch.stack(outs, dim=0)
```

```python
import numpy as np
from sklearn.svm import SVC
import pennylane as qml

dev = qml.device("default.qubit", wires=4)

def feature_map(x):
    for i in range(4):
        qml.RY(x[i], wires=i)
    for i in range(3):
        qml.CNOT(wires=[i, i+1])

@qml.qnode(dev)
def kernel_state(x):
    feature_map(x)
    return qml.state()

def kernel(x, y):
    # fidelity between embedded states (requires careful batching/caching)
    psi_x = kernel_state(x)
    psi_y = kernel_state(y)
    return np.abs(np.vdot(psi_x, psi_y))**2

def gram_matrix(X, Y):
    G = np.empty((len(X), len(Y)))
    for i, xi in enumerate(X):
        for j, yj in enumerate(Y):
            G[i, j] = kernel(xi, yj)
    return G

K_train = gram_matrix(X_train, X_train)
clf = SVC(kernel="precomputed").fit(K_train, y_train)
K_test  = gram_matrix(X_test,  X_train)
y_pred  = clf.predict(K_test)
```

# With QML TORCH, plug and play

```python
import torch
import qmltorch  # our library

model = torch.nn.Sequential(
    torch.nn.Linear(4, 4),
    qmltorch.QuantumLayer(
        n_qubits=4,
        encoding="angle",          # or "ar
        ansatz="ry_cnot",          # or "ha
        return_obs="pauli_z@0"     # readou
    ),
    torch.nn.Linear(1, 2)
)
# Gradients, batching, and device setup handled internally.
```

```python
from qmltorch import QuantumKernelSVM

clf = QuantumKernelSVM(
    n_qubits=4,
    feature_map="angle",     # or "amplitude", "zzfeature"
    cache=True, batch_size=64
)
clf.fit(X_train, y_train)
y_pred = clf.predict(X_test)
```

SO

WHATS

DIFFERENT ?



# So What's Differ

You must w ions).

Yo

You must gradients.

**Result:** rong. 👎



# So What's Different ? Normal Slide

## Today (PennyLane + PyTorch)

You must define a device (qml.device("default.qubit", wires=4)).

You must write your own encoding function (how raw numbers get turned into qubit rotations).

You must write your own ansatz (the sequence of quantum gates you'll apply).

You must wrap it in a QNode and tell PennyLane how it connects to PyTorch for gradients.

You must manually call the circuit for every sample in your batch.

You must handle gradient flow and make sure shapes match.

**Result: ~50–100 lines of code, lots of boilerplate, lots of things that can go wrong.** 👎

# So What's Different ?

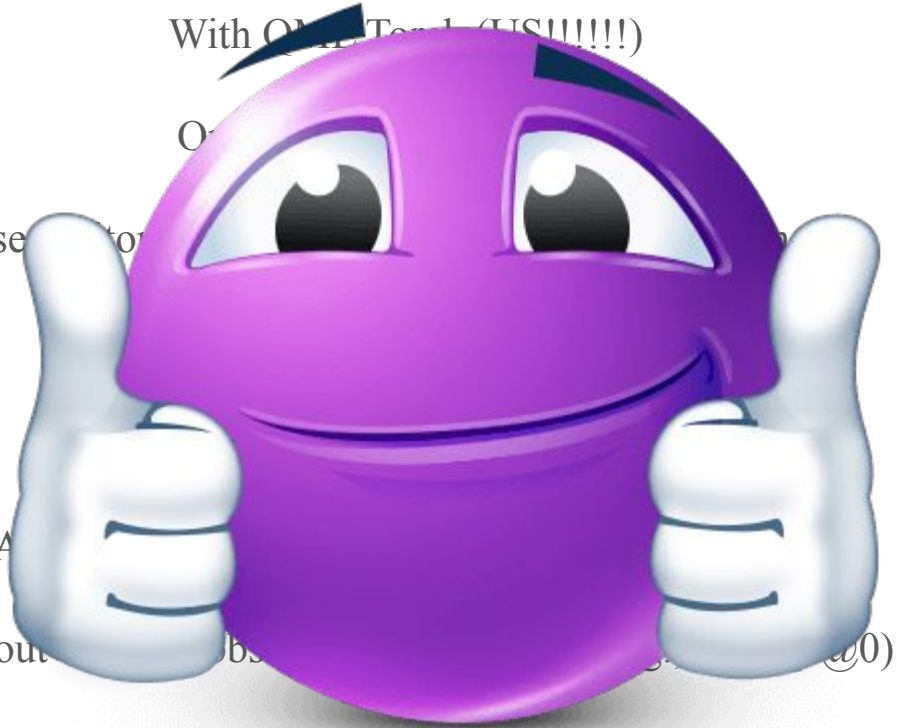
With Q[illegible] Tools (US!!!!!!)

O[illegible]

Use[illegible]to[illegible]

A[illegible]

Readout[illegible]bs[illegible]@0).



Batching, gradients, and device setup all handled internally.

👉 **Result: 5–10 lines of clean code, easy to drop in as a layer.**

# So What's Different ? (Normal Slide)

With QML Torch (US!!!!!!)

One line: import qmltorch.

Use qmltorch.QuantumLayer(...) just like nn.Linear.

Built-in options for:

Encoding → "angle" or "amplitude".

Ansatz → "ry", "ry_cnot", "hardware_efficient".

Readout → what observable to measure (e.g., pauli_z@0).

Batching, gradients, and device setup all handled internally.

👉 **Result: 5–10 lines of clean code, easy to drop in as a layer.**

**We don’t need a quantum computer in the lab. Libraries like PennyLane and Qiskit simulate qubits for us, and PyTorch handles the optimization. The heavy lifting is already written in C++/CUDA our job is to integrate it so it feels natural to ML developers.**

**Phase 0: Onboarding & Teaching (Weeks 0–2)**Install Python, PyTorch, scikit-learn, PennyLane, Qiskit, Workshop: what's a qubit, superposition, tiny circuits in code.,Mini exercise: train a quantum kernel SVM on the moons dataset. Everyone can run hybrid ML code locally.

## Phase 1: Core Toolkit (Weeks 1–6)

Build TorchQuantumLayer → PyTorch class wrapping PennyLane/Qiskit circuits.
Build **QuantumKernel** → sklearn-style kernel with caching + batching.
Add infrastructure: Docker, GitHub Actions CI, unit tests.
**Output:** First working Python library with tests.

## Phase 2: Case Studies (Weeks 7–16)

Apply QML Torch to real datasets:
Cybersecurity (intrusion detection).
Optimization (traveling salesman).
NLP (fake news).
Recommenders (MovieLens).
ioinformatics (codons/proteins).
Train classical vs hybrid models, compare metrics.
**Output:** Reproducible benchmark results.

## Phase 3: Documentation & Outreach (Weeks 17–24)

Write tutorials + API docs with MkDocs.
Jupyter notebooks (Intro to Qubits → Case Studies).
Streamlit demo app (upload dataset, test classical vs hybrid).
Outreach slides + recorded demos.
Package release: PyPI + Docker Hub.
**Output:** Polished toolkit with code, docs, and demo app.

## ✅ What We Are Doing

Building a Python library (PyTorch layers + sklearn kernels).

Running applied case studies (cybersecurity, NLP, optimization, recommenders, bio).

Providing reproducible tools (Docker, CI/CD, unit tests, CLI).

Writing tutorials, docs, notebooks, and a Streamlit demo app.

Benchmarking classical vs hybrid models honestly.

## ❌ What We Are Not Doing

Inventing new physics or simulators.

Training large-scale quantum models (hardware is too limited).

Simulating >30 qubits (RAM constraints).

Promising that quantum always outperforms classical.

Spending money on quantum hardware (we use simulators + free cloud tiers)

# Workflow

**Weekly Meetings**: 5-6 PM every Wednesday in Woodward 335, any changes will be announced on discord

**Lab/Office Hours:** Mondays 3:30-5:00 PM and Fridays 12:30 - 3:00 PM also DM me for specific timings if these do not work.

**Tools we will use:**

- Discord- Main communication
- Github- Version control & Project management
- Docker (maybe) if environmental and setup issues arise

I tried to answer a lot of questions on the github, so please go check it out it will be in the FAQ asked questions and whatnot folder

THE WILLIAM STATES LEE COLLEGE OF ENGINEERING

# SKILLS

- Python (Main Programming Language)
- PyTorch (Neural Networks & Training)
- scikit-learn (Classical ML & Kernels)
- PennyLane / Qiskit (Quantum Circuits & Simulation)
- Git & GitHub (Version Control + Collaboration)
- Docker (Optional Reproducibility)
- Documentation (MkDocs, Jupyter Notebooks)

**Everyone is welcome no prior quantum or ML background required. Just be curious and ready to learn!**

CHARLOTTE
THE WILLIAM STATES LEE
COLLEGE OF ENGINEERING